

# Demo v6.2 流水线

## Demo v6.2 流水线：23 个设计问题的代码答案

本文按设计评审中的 23 个问题，说明 Demo v6.2 每个阶段由哪个模块负责、数据如何流动，以及出错时在哪里终止。每个答案末尾都给出“源码证据”，引用格式为 文件::函数/类/方法；设计文档只作为语义补充，不替代实现证据。行号链接对应当前 single-camera 工作区源码。

整体运行结构如下：main.py::main 是总控入口，它启动 main\_data\_processing.py 子进程。后者由多个 mdp\_demo\_\* mixin 组成，负责 摄像头、分割、跟踪和 warm-up。总控进程持续读取摄像头进程写出的 frames.jsonl，通过 chunk\_\* 模块生成在线 chunk，同时通过 shape\_prior\_\* 模块生成 SAM3D shape prior，最后启动一个下游消费者：visualize\_track.py，或 Phystwin\_shen 的 scripts/run\_online\_full\_pipeline.py supervisor。Demo 只直接管理这个 supervisor；Stage 1、可选 Stage 2、train 和一个合并 HTML viewer 由外部 wrapper 创建并继承同一个进程组。生命周期事件写入 pipeline\_status.jsonl；Q23 会区分“已经实现的状态写入”和“默认 viewer 尚未显示的部分”。

六个 mdp\_demo\_\* mixin 都继承 annotation-only 的 mdp\_demo\_contract.\_DemoRuntimeContract。该 contract 集中声明跨 mixin 共享的 属性和方法签名；组装后的 MRO 把六个真实实现放在 contract stub 之前，所以它 只改善类型检查和 IDE 导航，不增加另一条运行路径，也不创建额外状态。

### 摄像头与逐帧 I/O（Q1–Q6）

1. 摄像头在哪里启动？正式编排路径中，main.py::main 先调用 main\_subprocess.build\_main\_data\_processing\_command，再用 subprocess.Popen 启动 main\_data\_processing.py。子进程入口构造 MainDataProcessingDemo；它继承的 \_LifecycleMixin.run 调用 main\_warmup.prepare\_runtime\_services\_and\_source，后者按输入模式选择 mdp\_capture\_source.\_start\_realsense\_pipeline (live) 或 RecordedRgbdfFrameSource (fake-live)。

正式编排的设备由 config/default.yaml 中 camera.camera\_serials 指定，默认是 ["239222300740"]。列表 schema 为以后多相机扩展保留，但 main\_options.resolve\_camera\_serials 当前要求 恰好一个非空 serial；可重复的 --camera-serial 会整体覆盖配置列表。main\_subprocess.build\_main\_data\_processing\_command 将唯一 serial 传给 子进程 --serial，\_start\_realsense\_pipeline 再执行 config.enable\_device(serial)。table\_calibrate\_metadata.json 也记录该 serial 为 table-calibration reference。

这个“一台指定相机”的约束只覆盖由 main.py 启动的正式路径。若直接运行 main\_data\_processing.py 且不传 --serial，utils.camera.resolve\_serial 仍会选择排

序后的第一个 D400。此前 D405 缺少所需 RGB profile 的报错是实机验证结果，不是 Python 源码主动抛出的 固定错误。

### 源码证据：

- [main.main](#) → [build\\_main\\_data\\_processing\\_command](#) → [main\\_data\\_processing.main](#) → [\\_LifecycleMixin.run](#)。
- [prepare\\_runtime\\_services\\_and\\_source](#) 包含 replay/live 分支；  
[\\_start\\_realsense\\_pipeline](#) 绑定设备并 启动 RealSense pipeline。
- [resolve\\_camera\\_serials](#)、[main\\_cli.build\\_parser](#)、[camera.camera\\_serials](#) 和 [table\\_calibrate\\_metadata.json](#) 共同 证明正式路径的 serial 合同。
- [list\\_d400\\_serials](#) 返回排序列表，[resolve\\_serial](#) 在未指定时取 `serials[0]`；实机 结果记录在 [2026-07-09-demo-v6-2-refactor.md](#)。

## 2. 摄像头线程在哪里创建？ `_LifecycleMixin._start_threads` 组装 worker 列表，并统一创建 `daemon=True` 的 `threading.Thread`。正式 strict 路径包含 `capture`、`seg`、`processed-frame`、`lossless tracker` 和 `pair-output worker`。

`pcd_mode=masked` 只允许这条 strict 路径：参数校验要求 TAPNext++ tracker 且 `track_mode != none`。无 tracker 只允许 `pcd_mode=none` 的纯 `capture/depth isolation`；旧的 `latest-frame _pcd_worker` 已删除。`processed-frame worker` 对正式 5 FPS 帧建立完整 world-space dense grid，按  $0.2 < \text{depth} < 1.5 \text{ m}$  与固定 PT radius rule 清理二维 mask，再从同一份 processed mask 选择 runtime PCD。它不执行点数 cap、table-Z 删除或 mask erosion；prepared writer 只打包该结果，不再次运行 PT。

`seg` 和 `lossless tracker` 会在同一个进程、同一个 CUDA device 上并发执行。EdgeTAM 的 reduce-overhead 会在第 2 次 model call 录制 CUDA graph；TAPNext++ 第一次拿到 mask 时才构造 CUDA model，并在参数初始化时使用 RNG。为避免两者重叠，live 与 replay 都会在发布 frame 0 后等待完整的首个 PCD/tracker pair，再释放 frame 1。这个启动 handshake 保留后续并发与 compile 性能，同时满足 CUDA graph capture 期间不能有其他线程 CUDA 工作的约束。`_capture_recording_worker` 不是另一条线程；replay 模式下，capture 线程进入 `_capture_worker` 后同步调用它。

shape-prior warm-up 另由 `ShapePriorWarmupManager.maybe_submit` 创建一条 shape-prior-warmup daemon thread；它不在 `_LifecycleMixin._threads` 列表中。

### 源码证据：

- [\\_LifecycleMixin.\\_start\\_threads](#) 定义 worker targets，并在一个循环里调用 `threading.Thread(..., daemon=True)`。
- [\\_CaptureMixin.\\_capture\\_worker](#) 的 replay 分支 直接调用 [\\_capture\\_recording\\_worker](#) 后返回。
- [ShapePriorWarmupManager.maybe\\_submit](#) 单独创建 shape-prior warm-up 线程。

3. **进程和线程如何协作？** 外层是多进程：总控进程启动 camera 子进程；可选 visualizer、Phystwin\_shen full-pipeline supervisor 也是总控的直接子进程。supervisor 再按配置启动一个合并 HTML viewer、Stage 1、可选 Stage 2 和 train；这些 child 不创建新 session，因此继承 supervisor 的进程组。shape-prior 不是一条与 camera 并列、常驻的单一进程：camera 进程先启动 warm-up thread，ShapePriorLocalClient.request\_shape\_prior 再顺序调用各阶段子进程。

camera 进程内部的 strict 数据流是：capture 同时写 capture\_slot 和 lossless\_frame\_queue；seg 将 raw mask 写到唯一 lossless\_mask\_queue；processed-frame worker 完成 depth/world-grid/PT 后，把同一 canonical frame 交给 runtime PCD 与 lossless\_processed\_frame\_queue 中的 tracker。PCD 与 tracker 结果由 SameSeqPairer 按相同 seq 配对，再进入 pair-output queue。PairedBuildResult.\_\_post\_init\_\_ 在对象构造边界再次要求 PCD packet、其 source mask、tracker packet 和 pair 本身四个 seq 完全相同；旧 PairedRenderPacket/paired\_render\_slot 中转层已经删除。pair-output worker 直接发布验证后的 build result。OrderedPacketQueue 保序且不静默覆盖，LatestSlot 只保留最新值。stop\_event 管理 LifecycleMixin.\_threads 中的 workers；shape-prior manager thread 不检查该 event，因此不能笼统说“所有线程都由 stop\_event 退出”。

#### 源码证据：

- [main.main](#)、[launch\\_phystwin\\_shen](#) 和 [ShapePriorLocalClient.request\\_shape\\_prior](#) 给出 Demo 侧进程边界；full-pipeline wrapper 的 child 顺序由外部 scripts/run\_online\_full\_pipeline.py 定义；shape-prior 阶段冷启动由 [run\\_stage](#) 调用 subprocess.run。
- [\\_publish\\_capture\\_packet](#)、[OrderedPacketQueue](#)、[SameSeqPairer](#) 和 [LatestSlot](#) 给出实际的线程间数据合同。
- [PairedBuildResult.\\_\\_post\\_init\\_\\_](#) 是 strict pair 的最终同序号校验；[DemoRuntimeContract](#) 声明跨 mixin 合同。
- [LifecycleMixin.stop](#) 设置 stop\_event 并 join \_threads；shape-prior thread 的入口是 [ShapePriorWarmupManager.\\_run](#)。

4. **RealSense RGB 和 depth 的 FPS 是多少？** 由 main.py 启动的正式路径默认以 **30 FPS** 采集 RealSense。build\_main\_data\_processing\_command 把外层 --camera-fps 传成子进程 --fps，\_start\_realsense\_pipeline 对所有启用的 color、IR 或 depth stream 都使用同一个 int(args.fps)。

strict live 输出采样由 LiveLatestFrameSampler 控制：直接输入是子进程的 --lossless-input-fps，外层由 resolve\_camera\_source\_replay\_fps 解析。未单独设置 --camera-source-replay-fps 时它回落到 --replay-fps，默认 **5 FPS**；设置 override 后，camera 取样频率可以与 chunk/metadata 使用的 replay\_fps 分开。采样器每隔 1/fps 发布当时的最新帧。直接运行 main\_data\_processing.py 的内层 --fps 默认仍是 60，因此 30 FPS 结论应限定为正式 main.py 编排默认。

这层采集频率不等于发布/物理频率。Demo 的 `replay_fps` 固定输出 **5 FPS** 均匀时间步；`Phystwin_shen configs/real.yaml` 同样使用 `FPS: 5`、`dt: 5e-5`、`num_substeps: 4000`，即每个数据帧模拟  $5e-5 \times 4000 = 0.2 \text{ s} = 1/5 \text{ s}$ 。因此不存在原先 30 FPS 物理时间比 Demo 快 6 倍的问题；RealSense 仍可在 30 FPS 获取最新输入。

PT mask refinement 同样位于这个 5 FPS formal 边界之后，所以 30 FPS 相机 帧只负责提供最新输入，不会逐帧承担 dense world-grid/PT 计算。

#### 源码证据：

- [config/default.yaml](#) 定义 `replay_fps: 5.0`，[camera.camera\\_fps](#) 定义 `camera_fps: 30`；[build\\_main\\_data\\_processing\\_command](#) 将后者传给子进程。
- [start\\_realsense\\_pipeline](#) 对每个启用 stream 都传 `int(args.fps)`。
- [resolve\\_camera\\_source\\_replay\\_fps](#)、[\\_CaptureMixin.\\_capture\\_worker](#) 和 [LiveLatestFrameSampler](#) 证明“固定 tick 取最新帧”的实际控制链。
- [mdp\\_cli.build\\_parser](#) 与 [mdp\\_constants.DEFAULT\\_FPS](#) 证明直接运行子入口的默认值是 60，而不是 30。

5. 每帧在哪里读取？live 模式由 `_CaptureMixin._capture_worker` 调用 `pipeline.wait_for_frames()`；`native-depth` 路径先 `align.process`，FFS 路径 读取 color 与左右 IR。replay 模式由 `_capture_recording_worker` 调用 `RecordedRgbFrameSource.read_packet` 加载录制 RGB-D/IR 文件。两条路径都 生成 `mdp_packets.FramePacket`。`fake-live` 的 `make_runtime` 保留相机几何和序列号，但令 `RealtimeCameraRuntime.pipeline=None`；只有 live runtime 持有需要停止的 RealSense pipeline。

`_publish_capture_packet` 总会写 `capture_slot`；正式 strict 模式还会同时 写 `lossless_frame_queue`，不是二选一。

#### 源码证据：

- [\\_CaptureMixin.\\_capture\\_worker](#) 包含 RealSense 读取、对齐、数组复制和 `FramePacket` 构造。
- [\\_capture\\_recording\\_worker](#) → [RecordedRgbFrameSource.read\\_packet](#) 是 replay 读取链。
- [RecordedRgbFrameSource.make\\_runtime](#) 和 [RealtimeCameraRuntime](#) 定义 fake-live 的无硬件 runtime 合同。
- [FramePacket](#) 定义逐帧包；[\\_publish\\_capture\\_packet](#) 明确先写 slot，再在 lossless 模式写 queue。

6. **frame id 和 timestamp 从哪里来？** `FramePacket.seq` 是 Demo 内部逻辑序号。live 路径用 `output_seq` 从 0 连续重编号；replay 路径把 `runtime_seq` 作为内部 seq。

fake-live 的来源字段更细：`source_timestamp_s` 来自录制 metadata 的 value；`source_frame_index` 是排序后 frame refs 中被选中的位置；原始 metadata key 则保存在

source\_step。当前 live 实现没有调用 RealSense

get\_timestamp()/get\_frame\_number(), 所以 live FramePacket 的三个 source\_\* 字段保持 None。原文“真实采集模式下来自摄像头”不受当前源码支持。

有值的 source\_\* 会一路写入 prepared frame 和 archive mapping。正式均匀 时间轴使用连续 online/output index 除以 fps, 而不是 raw source\_frame\_index / fps; source\_\* 只做 provenance。

源码证据:

- [CaptureMixin.capture\\_worker](#) 的 publish\_output\_packet 连续重编号, 且 live FramePacket(...) 未赋 source\_\*。
- [RecordedRgbDFrameSource.build\\_frame\\_refs](#) 与 [read\\_packet](#) 分别构造 step/timestamp, 再写 source\_timestamp\_s/source\_frame\_index/source\_step。
- [HeadlessCaptureWriter.write\\_pcd](#) → [prepare\\_phystwin\\_frame](#) 保留 provenance; [OnlineFrameArchive.archive\\_chunk](#) 写 online-to-source mapping。
- [ChunkDataWriter.commit\\_chunk\\_data\\_record](#) 维护连续 start\_frame/end\_frame; [write\\_chunk\\_from\\_rows](#) 用 row\_start/fps 和 row\_end/fps 生成均匀窗口时间。

## Warm-up (Q7–Q14)

7. **Warm-up 使用单帧还是一段视频?** 初始化使用一张帧。prepare\_segmentation\_warmup 取得一张 first\_frame, resolve\_initial\_mask\_bundle 只把这张图交给 SAM3.1, 生成 object、hand A、hand B 的 InitialMaskBundle。随后 \_seg\_worker 创建一个 EdgeTAM session, 并对同一 first\_frame 调用 \_run\_segmentation\_frame(..., add\_prompt=True)。这是一个 session 中的三个 identity, 不是三个 session。

源码证据:

- [prepare\\_segmentation\\_warmup](#) → [resolve\\_initial\\_mask\\_bundle](#) → [run\\_sam31\\_first\\_frame\\_mask\\_bundle](#) 只处理 first\_frame.color\_bgr。
- [SegWarmupMixin.seg\\_worker](#) 创建一个 EdgeTamVideoInferenceSession, 再调用 [\\_run\\_segmentation\\_frame](#) 并设置 add\_prompt=True。

8. **系统如何确认拿到的是 frame 0?** 正式 strict 路径不是靠 sentinel 单独保证, 而是靠 producer handshake 加 FIFO: live 首次发布时 output\_seq == 0, replay 首次显式调用 read\_packet(seq=0); 两者在发布首帧后都会等待 \_first\_frame\_segmented, 随后继续等待 \_lossless\_first\_pair\_published, 不会先把后续正式帧灌入。strict \_wait\_for\_first\_frame 从 OrderedPacketQueue 队首取帧, 因此得到 seq 0。

非 strict 分支才调用 capture\_slot.get\_latest\_after(-1); 它只保证返回 seq > -1 的当前最新帧, 不能单独证明一定是 seq 0。首帧只“添加 prompt 一次”, 但它还会继续用于 PCD、tracker、shape prior 和 warm-up anchor, 不能说整张帧只用于初始化。

源码证据:

- [wait for first frame](#) 展示 strict queue 与 non-strict latest-slot 两条分支。
- [CaptureMixin.capture\\_worker](#) 和 [capture\\_recording\\_worker](#) 都先发布 seq 0，再等待首帧分割 handshake。
- [OrderedPacketQueue.put/get](#) 保持连续 FIFO；[LatestSlot.get\\_latest\\_after](#) 则是 latest-wins。
- [SegWarmupMixin.seg\\_worker](#) 对首帧用 add\_prompt=True，后续帧统一用 False。

9. **Warm-up 期间后续到达的帧如何处理？** 后续帧继续进入同一个 EdgeTAM session，使用 add\_prompt=False，并继续生成 mask、PCD、tracker 结果和 input preview。只有正式 product row 受 gate 控制：chunk-ready anchor 已写且 shape-prior 仍为 pending/running 时，`_formal_chunk_rows_gated` 扣留 `frames.jsonl` row 和对应 tracker sidecar。

“期间 `frames.jsonl` 只有 frame 0”需要限定：anchor 前可能先写入无效 startup rows，chunk bridge 随后会修剪；gate 超时或 shape-prior 进入终态后也会解除。因此这句话只适用于正常的“anchor 已写、prior 仍在运行、gate 未超时”区间。

#### 源码证据：

- [SegWarmupMixin.seg\\_worker](#) 对后续帧调用 `_run_segmentation_frame(..., add_prompt=False)` 后仍发布 mask。
- [PairPublishMixin.publish\\_strict\\_pair](#) 在 PCD/tracker 已配对后只对落盘行应用 gate；`_formal_chunk_rows_gated` 定义其状态条件。
- [PcdMixin.write\\_headless\\_pcd\\_result](#) 在 gated 时返回；[HeadlessCaptureWriter.write\\_input\\_frame](#) 不经过该 gate。
- [trim\\_warmup\\_delayed\\_rows](#) 修剪无效 startup rows；[\\_headless\\_product\\_rows\\_gated](#) 实现超时解除逻辑。

10. **Warm-up 最耗时的步骤是什么？** 权威产物是 `capture/shape_prior_profile.json`。现在它把操作员等待时间拆成 三层，而不是只报告 submit 后的五个粗阶段：

1. `shape_prior_timing.pre_submit`：camera runtime start → frame 0 receive → EdgeTAM mask ready → PCD ready → shape-prior submit。它同时保留 frame-0 SAM3.1、EdgeTAM 和 PCD 的已有细分 timing。
2. `shape_prior_timing.critical_path`：case write → upscale → SAM3.1 RGBA export → SAM3D generate → align → sample → result finalize。每项有 start/end offset、wall duration 和子阶段 details。
3. 顶层 `warmup_shape_prior_ready_to_gate_open_ms` 与 `warmup_total_ms`：记录 prior ready 后写 capture 产物并真正打开 formal gate 的延迟，以及从 camera runtime start 到 gate open 的总等待。

当 shape-prior 产物已经写入且 formal gate 首次打开时，camera 子进程会在当前终端只打印一次醒目的 Warmup finished banner。它和 `STAGE_WARMUP_READY` 在同一个真实完成边界触发，不会在 submit 或后台计算尚未完成时提前显示。



shape\_prior\_timing.ranking 按关键路径 wall duration 排序，bottleneck 是本轮第一优化目标；accounted\_ms 和 unattributed\_ms 用于检查计时是否 闭合。这里的排名只描述本轮，不能用单次结果代替多轮 p50/p95。

三个预热子进程还会在 <shape-prior-

case>/shape/timing/{upscale,generate,align}.json 写 READY 与 completed 快照；sample 写 sample.json。父进程比较 READY wall time 与 GO wall time，给出 ready\_before\_go、ready\_lead\_ms 和 startup\_tail\_on\_critical\_path\_ms，因此可以区分“模型已经预热完成”和“GO 发出后仍在补模型加载”。profile\_snapshot\_to\_parent\_return\_ms 还把 最后一次 profile snapshot 后的 JSON flush/进程退出成本单独暴露出来。

子阶段拆分直接对应可优化动作：upscale 区分 model load/crop/inference/ PNG write；generate 区分 prepare/model load/pipeline run/GLB/PLY export；align 区分 input/render candidates/SuperGlue/PnP+scale/两段 ARAP/mesh export；sample 区分 mesh load/surface+volume sample/voxel dedup/pickle。

generate 的 SAM3D 调用固定关闭 with\_layout\_postprocess：Demo v6.2 只消费 GLB mesh 和 Gaussian，不使用 SAM3D 返回的 layout/pose 字段；随后独立的 shape\_prior\_align 才负责把 mesh 配准到 frame-0 观测。因而 layout pose 后处理不会决定最终对齐，只会增加 warm-up 关键路径。mesh postprocess 与 texture baking 仍保持启用，分别服务后续 mesh 对齐/采样和带纹理 GLB 导出。

修改前保存的一轮基线中，submit 后总计约 59.23 s：generate 29.44 s（49.7%）、align 14.26 s（24.1%）、upscale 12.27 s（20.7%）；三者合计 94.5%。camera start 到 submit 另约 15.94 s。因此下一轮详细 profile 应先 判断 generate 的模型加载还是 pipeline/export 最慢，再判断 align 的 render/match 与 ARAP，占比很小的二次 SAM3.1 segment 不是首要目标。

## 源码与实测证据：

- [shape\\_prior\\_timing](#) 定义 schema 校验、关键路径 闭合与 bottleneck 排名；[ShapePriorLocalClient.request\\_shape\\_prior](#) 聚合它。
- [run\\_sam3d\\_shape\\_prior](#)、[shape\\_prior\\_align.main](#)、[shape\\_prior\\_sample.main](#) 和 [image\\_upscale.main](#) 写子进程明细。
- [outputs\\_v6\\_1/capture/shape\\_prior\\_profile.json](#) 是修改前 59.23 s 基线；需要下一次运行才会生成新的详细 schema。

## 11. Warm-up 完成后保留哪些状态和文件？\_seg\_worker 在其生命周期内保留

SegmentationWarmupState、InitialMaskBundle 和唯一的 EdgeTAM session；三个 identity 在同一次 add\_inputs\_to\_inference\_session 中注册。shape-prior manager 成功后还在 self.\_result 中保留 ShapePriorResult。

磁盘上会保留 offline-style shape-prior case、配置路径下的 shape\_prior/points.npz、capture 下的 shape\_prior/points.npz、<case>/shape/matching/final\_mesh.glb，以及

shape-prior sampling 生成的 <case>/final\_data.pkl。

#### 源码证据:

- [InitialMaskBundle](#) 与 [SegmentationWarmupState](#) 的实例由 [\\_SegWarmupMixin.seg\\_worker](#) 持有；同一 worker 只构造一个 session。
- [ShapePriorWarmupManager.run](#) 成功时写 `self._result`，[ready\\_result](#) 读取它。
- [write\\_shape\\_prior\\_case](#) 与 [ShapePriorLocalClient.request\\_shape\\_prior](#) 写 case 和配置的 `points.npz`；[HeadlessCaptureWriter.write\\_shape\\_prior\\_result](#) 写 capture 副本。
- [shape\\_prior\\_align.main](#) 导出 `final_mesh.glb`；[shape\\_prior\\_sample.main](#) 写 case `final_data.pkl`。

12. **Warm-up 状态如何校验？** `resolve_initial_mask_bundle` 校验 controller/object mask 与输入帧尺寸；`_union_masks` 拒绝“没有任何 instance mask”和同一 label 内形状不一致，但不会把“一张全 false mask”自动视为缺失。`split_controller_hand_instances` 必须得到两只非空、可分离的手。

canonical processed-frame 边界校验 c2w，并在固定 PT 后立即拒绝空 object 或 controller mask。`write_shape_prior_case` 只校验并写入同一份 cleaned mask/dense grid，不重算 PT，也不会借用 object point 伪造 controller。

#### 源码证据:

- [resolve\\_initial\\_mask\\_bundle](#)、[\\_union\\_masks](#) 和 [split\\_controller\\_hand\\_instances](#) 给出 frame-0 mask 校验。
- [PcdMixin.build\\_processed\\_frame\\_result](#) 构建 canonical frame 并对空类别 fail fast；[write\\_shape\\_prior\\_case](#) 直接消费该 frame。
- [\\_shape\\_points\\_for\\_chunk](#) 只检查 surface+interior 总数是否大于 0；不存在更高的通用最小点数门槛。

13. **Warm-up 出错后如何处理？** 冷启动 shape-prior stage 由 `_run_stage` 使用 `subprocess.run(..., check=True)`；预热 worker 也会在非零 return code 时抛 `CalledProcessError`。但 shape-prior 总控本身是 camera 进程内的 daemon thread；`ShapePriorWarmupManager._run` 会捕获 stage 异常，将 profile 置为 failed，而不是直接走 camera worker 的 fatal hook。

segmentation 等正式 workers 的异常会进入 `_record_fatal_worker_error`：记录第一条 fatal、写 `fatal_error` 状态、设置 `stop_event`，最后让 `MainDataProcessingDemo.run` 返回 2。shape-prior 失败时，status 不再是 pending/running，row gate 解除；失败 profile 写入 capture metadata，chunk bridge 的 `_shape_points_for_chunk` 看到 failed 后立即抛错，避免无限等待。

#### 源码证据:

- [\\_run\\_stage](#) 和 [\\_run\\_prewarmed\\_stage](#) 给出子阶段非零退出的 异常路径。



- [ShapePriorWarmupManager.maybe\\_submit](#) 创建 thread;  
[ShapePriorWarmupManager.run](#) 捕获 异常并写 STATUS\_FAILED。
- [\\_record\\_fatal\\_worker\\_error](#) 与 [LifecycleMixin.run](#) 给出 camera worker 的 fatal/exit 路径。
- [\\_formal\\_chunk\\_rows\\_gated](#)、[\\_maybe\\_write\\_shape\\_prior\\_headless\\_result](#) 和 [\\_shape\\_points\\_for\\_chunk](#) 给出 shape-prior failed → metadata → bridge exception 的真实传播链。

14. 正式时间线从哪里开始？ 成功路径中，第一个 chunk-ready row 占据 online/output frame 0 的 warm-up anchor；anchor 后、shape prior 尚为 pending/running 的 rows 被扣留。shape prior 进入 READY 后第一条未被 gate 的 row 紧接为 online frame 1。OnlineFrameArchive 连续编号，同时在 enhance\_metadata.json 保留原 seq 和 source mapping。

这里也有失败边界：gate 会在 timeout 或 failed/disabled 终态解除，所以“READY 后成为 frame 1”只描述成功路径；失败路径随后应由 bridge 报错。

源码证据：

- [\\_formal\\_chunk\\_rows\\_gated](#) 定义 anchor 后的 gate；  
[PcdMixin.write\\_headless\\_pcd\\_result](#) 只让 controller ≥ 30 且 object > 0 的 row 取得 anchor，并记录首次 formal seq。
- [\\_trim\\_warmup\\_delayed\\_rows](#) 保留首个 chunk-ready row；  
[OnlineFrameArchive.archive\\_chunk](#) 用 online\_start\_frame + local\_index 连续编号。
- [design\\_spec.md](#) 记录 frame 0/1 接缝与 hold-still 约定，但实现依据是上述函数。

## Chunk 组装、跟踪与过滤 (Q15–Q19)

15. **Chunk 如何组装？** stream\_chunk\_data\_from\_headless\_capture 每读到一条 frames.jsonl row，就用 \_prepared\_frame\_from\_row 加载它引用的 canonical prepared NPZ，并将 row/frame 同步追加到两个 buffer。达到 chunk\_size 后先形成 pending\_window；下一条 row 通常作为 borrow/lookahead frame 到达后才 materialize，capture 结束时则无 borrow flush。borrow 只参与上一窗口尾帧的 motion verdict，不进入其发布数组。

\_chunk\_data\_window\_from\_prepared\_frames 校验所有帧共享 query schema，逐帧 收集 processed mask，并将 track、visibility、PCD points/colors 沿时间维 stack。RGB 不在这里 stack，而是稍后由 OnlineFrameArchive.archive\_chunk 逐帧写 PNG；query\_points\_yx 只做共享一致性校验。当前路径不会从 legacy sidecar 重建数据。

源码证据：

- [stream\\_chunk\\_data\\_from\\_headless\\_capture](#) 负责 row/frame 双 buffer、pending\_window 和 borrow-row 触发；其嵌套 [\\_materialize\\_pending](#) 定义在同函数内。
- [\\_prepared\\_frame\\_from\\_row](#) 要求 prepared\_phystwin\_frame\_path；缺失或文件不存在立即失败。
- [\\_chunk\\_data\\_window\\_from\\_prepared\\_frames](#) 检查 query 一致性、收集 mask，并 stack track/visibility/PCD；[write\\_chunk\\_from\\_rows](#) 将 RGB-D 交给 archive。

16. **Chunk 大小在哪里配置？** main\_options.resolve\_chunk\_frame\_count 优先使用显式 --chunk-frame-count；否则计算  $\text{round}(\text{replay\_fps} \times \text{chunk\_seconds})$ ，并要求结果大于 0。默认配置为 5 FPS × 1 秒 = **5 帧**。结果传给 chunk stream，并保存为 ChunkDataWriter.chunk\_size；writer 再次校验正数，并把它写入 manifest 和 static metadata。

**源码证据：**

- [config/default.yaml](#) 定义 5 FPS，[chunking.chunk\\_seconds](#) 定义 1 秒；[main\\_cli.build\\_parser](#) 暴露 override。
- [resolve\\_chunk\\_frame\\_count](#) 实现 override/乘法/正数 校验；[main.main](#) 把结果传入 chunk stream。
- [ChunkDataWriter.\\_init](#) 再次校验并保存 self.chunk\_size；[write\\_manifest](#) 发布它。

17. **Chunk 按时间还是按帧数关闭？** 窗口边界严格按 **row/frame 数量**：每次 append 后，buffer 未满足就 continue，达到 chunk\_size 才关闭。window\_closed\_wall\_s 只是遥测，不参与边界判断。live 路径关闭完整窗口后，通常仍需等下一帧作为 borrow 才 materialize/commit，因此“按帧数关闭”不等于“第 chunk\_size 帧到达 即发布”。

**源码证据：**

- [stream\\_chunk\\_data\\_from\\_headless\\_capture](#) 用  $\text{len}(\text{row\_buffer}) < \text{chunk\_size}$  判断是否继续累积。
- 该函数中的 pending\_window 与 [\\_materialize\\_pending](#) 证明 borrow-row 发布延迟；[write\\_chunk\\_from\\_rows](#) 只把 window\_closed\_wall\_s 写入 telemetry。

18. **Chunk 组装后如何执行跟踪？** 唯一的实时 chunk-stream 入口为整个 session 创建一次 tracking.TrackingRuntime，并把同一实例传过 [\\_write\\_chunk\\_from\\_rows](#) 到 [\\_chunk\\_data\\_window\\_from\\_prepared\\_frames](#)。后者调用 [\\_track\\_input\\_with\\_session\\_query\\_schema](#) → tracking.build\_window\_observations → TrackingRuntime.process\_window。

chunk 0 的 [\\_freeze\\_identity](#) 冻结 controller anchors、object columns、query schema 和 neighbor table；后续窗口由 [\\_check\\_frozen\\_identity](#) 校验。process\_window 用 ~ctrl\_usable 表示语义上的 temporary-invalid（没有单独命名的状态数组），再调用 [\\_recover\\_anchor](#) 和 Kabsch\_rigid\_transform 做局部刚性恢复。若单独调用 window

builder 而不传 runtime，它会临时新建实例；“整个 session 一个 runtime”只保证在公开 stream 主路径中成立。

#### 源码证据：

- [stream\\_chunk\\_data\\_from\\_headless\\_capture](#) 在循环外创建一个 TrackingRuntime。
- [\\_track\\_input\\_with\\_session\\_query\\_schema](#) 调用 [build\\_window\\_observations](#)；  
[\\_chunk\\_data\\_window\\_from\\_prepared\\_frames](#) 调用 [TrackingRuntime.process\\_window](#)。
- [\\_freeze\\_identity](#)、[\\_check\\_frozen\\_identity](#)、[\\_recover\\_anchor](#) 和 [\\_rigid\\_transform](#) 给出冻结与恢复细节。

19. 跟踪后还会做哪些过滤？ tracking.motion\_consistency 执行动作一致性过滤：半径 0.01 m、至少 5 个邻居（radius query 未排除自身）、相似阈值 0.005 m，至少 50% 邻居同意。

depth-validity **mask refinement** 与固定的 3D radius-outlier mask refinement（radius=0.01 m、nb\_points=40）在 processed-frame worker 中按顺序执行一次；tracker、runtime PCD、shape prior、prepared writer 和 chunk 侧只消费同一结果，不再次做 radius refinement。不过 tracking.build\_window\_observations 仍会在 track pixel 采样时重新计算逐 query 的 depth-valid，因此不能泛称“所有 depth-validity 判断只执行一次”。

tracking 之后，AsapRuntime.augment\_window 以 visibility、motion validity、finite 和 nonzero 联合判定无效 object 条目并回填。\_deform\_frame 在约束太少或结果非有限时复用上一帧 mesh vertices，这就是 design\_spec\_v6\_1.md 所保留的 silent-freeze 行为。

#### 源码证据：

- [tracking.py](#) 常量 与 [motion\\_consistency](#) 给出 0.01/5/0.005/50% 规则。
- [PcdMixin.build\\_processed\\_frame\\_result](#) 依次调用 [apply\\_depth\\_validity\\_to\\_mask\\_frame](#) 和 [apply\\_radius\\_outlier\\_to\\_mask\\_frame](#)；  
[prepare\\_phystwin\\_frame](#) 只验证和打包；[build\\_window\\_observations](#) 另做 query-level depth-valid。
- [AsapRuntime.augment\\_window](#) 计算 valid\_now 并回填；[AsapRuntime.deform\\_frame](#) 实现 silent freeze。

## 训练侧 schema、manifest 与读取起点（Q20–Q22）

20. 训练侧会收到什么数据？ Demo 生产端把每个窗口写成

online\_data/chunks/chunk\_{id:06d}.pkl。固定 metadata 包括 case\_name、chunk\_id、start\_frame、end\_frame、source\_frame\_indices；source\_timestamps\_s 仅在有价值时写入。data\_keys.REQUIRED\_TIME\_KEYS 定义五个生产端必需时序键：object\_points、object\_colors、object\_visibilities、object\_motions\_valid、controller\_points。生

产端通用 schema 把 asap\_surface\_points、asap\_interior\_points、controller\_proxied 等列为 optional；并不存在名为 recovery\_mask 的字段。

但当前配置的 Phystwin\_shen consumer 会把 asap\_surface\_points/asap\_interior\_points 提升为必需字段；默认 asap\_augment=True 会生产它们。因此“ASAP 可选”只描述 Demo writer 的通用 schema，不描述当前 trainer 的更窄合同。

同一 session 还生成 online\_data/color/0/{k}.png、online\_data/depth/0/{k}.npy (uint16 mm)、calibrate.pkl、metadata.json、enhance\_metadata.json，以及前缀聚合后的 data/final\_data.pkl。

#### 源码证据：

- [REQUIRED\\_TIME\\_KEYS/OPTIONAL\\_TIME\\_KEYS](#) 定义生产端键；[build\\_chunk\\_data\\_record](#) 定义 chunk metadata 与 TIME\_KEYS 切片。
- [ChunkDataWriter.commit\\_chunk\\_data\\_record](#) 写 chunk\_{id:06d}.pkl；[\\_append\\_static\\_data](#) 聚合 data/final\_data.pkl。
- [OnlineFrameArchive.archive\\_one\\_frame](#)、[\\_initialize\\_calibration](#)、[\\_write\\_metadata](#) 和 [\\_write\\_enhance\\_metadata](#) 给出 RGB-D archive 布局。
- 当前外部 checkout 5b8c071 的 [OnlineFrameBuffer.\\_validate\\_chunk\\_shapes](#) 将两个 ASAP key 列为 required；[main\\_cli.build\\_parser](#) 把 asap\_augment 默认设为 true。

#### 21. Manifest 何时更新，如何保证读者不会看到半成品？ 正常提交顺序是：

OnlineFrameArchive.archive\_chunk 先写本 chunk 的 RGB-D 帧文件；ChunkDataWriter.commit\_chunk\_data\_record 再原子写 chunk pickle，原子更新聚合 final\_data/metadata，最后原子更新 online\_data/manifest.json；commit 返回后才调用 OnlineFrameArchive.publish\_metadata 推进 archive 的 metadata.json 与 enhance\_metadata.json。所以读者一旦从 manifest 看到新 committed chunk，对应 chunk 与帧文件已经存在；archive frame\_num 只推进到已 commit 前缀。

atomic\_pickle\_dump/atomic\_json\_dump 都采用临时文件、flush、fsync 和 os.replace。RGB PNG 也 fsync 后 replace；depth NPY 使用 temp+replace 但没有显式 fsync，所以源码保证“不会看到半写文件”，不能夸大成“所有 archive 文件都具备相同的断电持久性”。正常结束写 finished；materialize/commit try 块内的失败写 failed。更早的 prepared-frame 加载失败不在该 try 块内，因此不能声称任何异常都必然把 manifest 从 recording 改成 failed。

#### 源码证据：

- [\\_write\\_chunk\\_from\\_rows](#) 明确执行 archive\_chunk → commit\_chunk\_data → publish\_metadata。
- [ChunkDataWriter.commit\\_chunk\\_data\\_record](#) 的顺序是 atomic chunk → aggregate → counters → [\\_write\\_manifest](#)。

- [OnlineFrameArchive.archive\\_chunk](#) 与 [publish\\_metadata](#) 说明 frame files 与 committed metadata 的关系。
- [atomic\\_pickle\\_dump/atomic\\_json\\_dump](#) 和 [\\_atomic\\_write\\_bytes](#) 给出原子写细节；[stream\\_chunk\\_data\\_from\\_headless\\_capture](#) 给出 finished/failed 的实际覆盖边界。

22. 训练侧从什么时候开始读取？ main.main 内嵌的 \_maybe\_start\_phystwin\_shen 在 warm-up 启用时等待 shape\_prior/points.npz 出现后启动；warm-up 禁用时以 warmup\_disabled\_immediate 立即启动。Demo 只启动一个 scripts/run\_online\_full\_pipeline.py supervisor，并显式传入 --online\_dir <base\_path>/online\_data 以及本地 config/default.yaml::phystwin\_shen 的每个 runtime 叶子。外部 pipeline YAML 不再是 Demo 参数的维护源。Stage 1/2 各自的 max\_online\_chunks、cma\_popsizes、zero\_order\_backend 和 sim\_force\_mode 也由本地 YAML 显式传入；当前 Stage 1 是 2/4/boba/gather，Stage 2 是 10/4/boba/gather。batch\_size/segment\_len/segment\_stride 遵循 Phystwin 原生的 common-then-stage 继承语义，并通过 stage-specific CLI 显式传递。当前 common 不提供这三个默认值，Stage 1 使用 2/10/10，train 使用 5/30/30；禁用的 Stage 2 可省略，若启用且没有自身值或 common 默认值，Demo 会在 camera 启动前失败。

supervisor 在 demo\_2\_max 中运行；外部 YAML 的 python: null 令 Stage 1、Stage 2、train 和 viewer 继承 supervisor 的同一个 Python。wrapper 先启动 cma\_viewer.source=all 的单个合并 viewer，再顺序运行 Stage 1、可选 Stage 2 和 train；独立 train\_viewer 保持关闭。Demo 在启动 supervisor 前只清理这个 viewer 的 endpoint，默认是 127.0.0.1:8765。

supervisor、Stage 1/2 和 train 的合并 stdout/stderr 同时实时转发到 Demo 启动终端（每行前缀 [phystwin\_shen]）并保留到

<base\_path>/phystwin\_shen/online\_full\_pipeline.log。Demo 显式设置 PYTHONUNBUFFERED=1，避免外部 Python stage 因 pipe 重定向延迟输出。当前默认 viewer 使用 --quiet，它们自己的输出仍由外部 wrapper 静默处理。

顺序读取逻辑位于外部 Phystwin\_shen checkout：OnlineChunkReader.load\_new\_chunks 从 last\_loaded\_chunk + 1 顺序读到 manifest 的 latest\_committed\_chunk；wait\_for\_initial\_frames 会先等到至少各 stage 自己的 segment\_len 帧才创建 simulator/trainer。当前 chunk 是 5 帧，Stage 1 需要 10 帧（2 chunks），train 需要 30 帧（6 chunks）。train.stop\_when\_finished: true 时，trainer 以 manifest 的 finished 为停止条件，完成并保存观察到 finished 的 terminal iteration；设为 false 时严格跑 iterations。无论是哪一种，Demo 正常路径最终都会等待 supervisor 返回，只有 return code 0 才算完整 pipeline 成功。

源码证据：

- [main.main::\\_maybe\\_start\\_phystwin\\_shen](#) 给出 points-ready/disabled 两个 trigger；[launch\\_phystwin\\_shen](#) 只执行一次 Popen，[build\\_full\\_pipeline\\_command](#) 给出完整

显式 CLI。

- 外部 checkout 的 `train_online_warp.py::wait_for_initial_frames`、`qqt/data/online_stream.py::OnlineChunkReader.load_new_chunks` 和 `qqt/engine/trainer_warp.py::InvPhyTrainerWarp.train_online_batched` 给出等待、顺序读取、terminal save 和 stop-when-finished 行为。

## 在线流水线状态可视化 (Q23)

23. 如何看到流水线当前在做什么，以及 **warm-up** 是否失败？ `PipelineStatusWriter.emit` 把 `t/source/stage/detail/ok` 追加到 `<base_path>/pipeline_status.jsonl`，并吞掉所有写入异常，所以 telemetry 失败不会改变正式产品。当前实际 writer 只有两类：orchestrator 写 run start、chunk committed、Phystwin downstream start 和正常控制流末尾的 finished/fatal；camera 写 capture start、shape-prior submitted、warm-up ready，以及 startup/worker fatal。shape-prior 的 upscale/generate/align/ sample 子进程没有各自创建 writer，也没有逐阶段状态事件。

`viz_playback.run_side_by_side` 会读取最近 200 条事件，并调用 `viz_panels.draw_pipeline_status` 绘制状态条；任一 `ok=False` 或 `STAGE_FATAL` 事件会令状态条变红并显示错误。但当前默认配置是 side-by-side + sam3d-final-data，`visualize_track.run` 会分派到 `run_interactive_side_by_side`；该函数目前没有读取状态日志，也没有绘制状态条。因此当前可核验的查看方式是：直接查看 `pipeline_status.jsonl`，或使用会进入 OpenCV `run_side_by_side` 的 `rgb-overlay` 路径；不能声称默认正式 viewer 已经显示该状态条。

`main.main` 的外层 `try/except/finally` 覆盖 camera 启动、stream、supervisor launch 和最终 wait。shape-prior/materialization、camera、supervisor 非零退出或 Ctrl+C 任一失败都会写 terminal `STAGE_FATAL`，并以保存的 PGID 对整个 Phystwin\_shen 进程组执行 `SIGTERM`，超时后 `SIGKILL`。

显式 `--max-chunks N` 达标时，stream 已经提交完第 N 个 chunk 并持久化 terminal `manifest.status=finished`。编排器会在等待 Phystwin\_shen 继续训练之前立即、只打印一次：

```
#####  
collect finish  
#####
```

这个 banner 只表示采集完成，不表示整个 Demo 进程已经退出；Phystwin\_shen 尚未训练完时，主进程会继续等待它。

**Warm-up 实时 RGB 输入预览** (`mdp_warmup_preview.WarmupRgbPreview`，不是 tracking-chunk viewer)：无论 `downstream.mode` 选什么，camera 进程在 capture 启动时打开一个实时 RGB 输入窗口（直接读内存里的 `input_preview_slot`，零磁盘 IO），供操作员在 hold-still 期间确认取景/双手可见。生命周期：warm-up 正常结束（`WARMUP_FINISHED` banner 处；若 shape-prior warm-up 关闭则在 frame-0 seed 完



成处) 自动关闭; warm-up 失败/取消/提前退出经 `stop_event + stop()` 立即关闭。GUI 失败 (无显示环境) 只打一行日志并禁用, 绝不影响采集。开关: `--warmup-rgb-preview / --no-warmup-rgb-preview` (默认开, 编排器透传给 camera 子进程)。即使 supervisor leader 已退出, 遗留的 train/viewer child 仍按保存 PGID 清理。reader 仍只把 `manifest.status=finished` 当完成; 若 producer 写 failed, Demo 不等待 reader 自己理解 failed, 而是由上述异常路径终止整个 downstream group。

### 源码证据:

- [PipelineStatusWriter.emit](#) 是 best-effort append; [read\\_status\\_events](#) 容忍缺失文件和 torn last line。
- [main.main](#)、[LifecycleMixin.run/\\_record\\_fatal\\_worker\\_error](#) 和 [\\_maybe\\_start\\_shape\\_prior\\_from\\_pcd\\_result](#) 是全仓实际 emit call sites; shape-prior stage 文件没有 writer call site。
- [viz\\_playback.run\\_side\\_by\\_side](#) 调用 [draw\\_pipeline\\_status](#); [use\\_interactive\\_side\\_by\\_side](#) 与 [run\\_interactive\\_side\\_by\\_side](#) 证明默认 SAM3D viewer 绕过该绘制逻辑。
- [config/default.yaml](#) 定义默认 side-by-side + sam3d-final-data; [ShapePriorWarmupManager.\\_run](#) 与 [main.main](#) 给出 terminal fatal 与 downstream PGID 清理边界。